

RESEARCH ARTICLE

A new marker design for a robust marker tracking system against occlusions

Hugo Álvarez*, Ibai Leizea and Diego Borro

Ceit and Tecnun (University of Navarra), Manuel de Lardizábal 15, 20018 San Sebastián, Spain

ABSTRACT

Marker systems are a widely used optical tracking method that does not support occlusions. Thus, this paper proposes a new marker design to overcome the problem of marker occlusions. It is highly adaptable, because it can be used by any marker tracking system that uses its central area to codify the digital identification. Our proposal takes advantage of an untapped frame to place some textures that will be tracked during marker occlusion. In addition, these textures are customizable, which lets users make their own designs. Two tracking methods are combined to offer a robust tracking, updating the six degrees of freedom of the camera in real time. The first one is a fast technique based on temporal coherence, whereas the second one is a robust technique based on appearance, which is used as a recovery mode. Copyright © 2012 John Wiley & Sons, Ltd.

KEYWORDS

augmented reality; marker; tracking; occlusion

*Correspondence

Hugo Álvarez, Ceit and Tecnun (University of Navarra), Manuel de Lardizábal 15, 20018 San Sebastián, Spain.

E-mail: halvarez@ceit.es

1. INTRODUCTION

The aim of augmented reality is to add virtual objects to the real world, giving the impression that both worlds coexist. To achieve this goal, much effort has been made to obtain an accurate and robust tracking system based on computer vision. Markerless solutions use natural features [1,2] or a 3D model [3,4] to calculate the camera pose. These alternatives do not need environment adaptation, because they rely on natural features that are in the scene or lie on the surface of the model to be tracked. Thus, in some cases, a scene rich in texture is required, whereas in other cases, the 3D model must be known, which is not always easy to obtain. Marker tracking systems are an alternative to those problems, because they add special patterns to the scene [5–7]. Furthermore, they are faster, more accurate, and more robust than markerless tracking solutions. The main inconvenience is the environment adaptation, although in some cases, this adaptation is feasible and does not involve any obstacle at all. Besides the environment adaptation, marker occlusion is another shortcoming, as the system fails even if the marker is only slightly occluded (Figure 1, left). This produces an undesirable effect on users who lose the sense of realism. Because of this limitation, instead of using multiple markers so that there is always one marker visible [8], we have developed a new

marker design that deals with occlusions and updates the six degrees of freedom of the camera (Figure 1, right). Our proposal offers more robustness and does not need more environment modification.

We have chosen ARToolkitPlus [9] to introduce our ideas because it is a widely used noncommercial marker tracking system that uses markers that do not take advantage of their frame to codify information (Figure 2, left). Notice, however, that our method can be adapted to any marker that does not codify information on its frame. We have placed textured patches along the frame of the marker to have more visible features (with known 3D coordinates) during marker occlusion. Moreover, the content of these textures is customizable, which lets users design their own markers or expand marketing opportunities. In addition, the different occlusion states provide us with additional information that can be used to implement some human–machine interfaces.

The rest of the paper is organized as follows. First, some works related to marker occlusion will be enumerated. Afterwards, the choice of the new marker design will be justified. Then, all steps involved in the tracking will be described. Additionally, some experiments will demonstrate the performance of the proposed method. Finally, conclusions and future works will be cited.

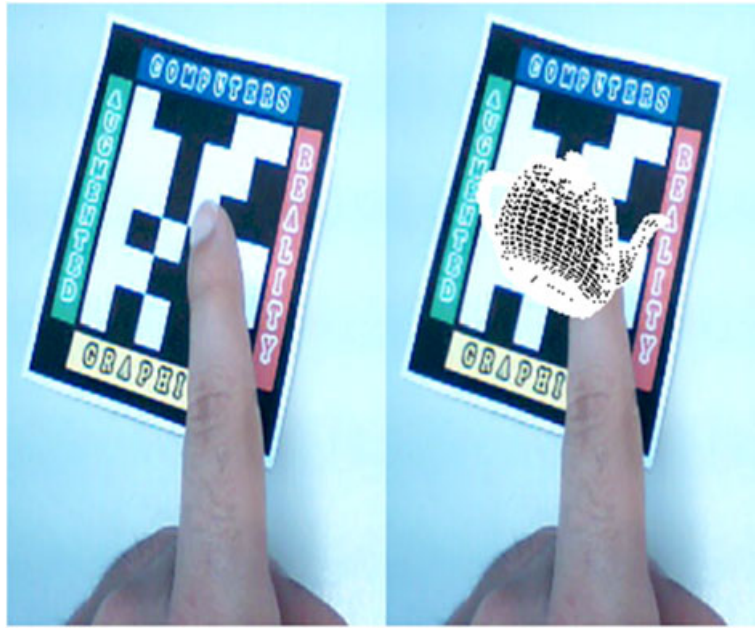


Figure 1. An example of failure when the marker is occluded (no virtual teapot is rendered) (left), and the output that is obtained using the proposed occlusion module (right).



Figure 2. The new marker design that is proposed in this paper. Initial version of the ARToolkitPlus BCH marker (left), an evolution that uses its frame to add some textures (middle), and the final version using color codification to accelerate the feature matching step (right). These textures are only an example, as they are customizable by the user.

2. RELATED WORK

Some authors place multiple markers in the scene to solve the marker occlusion problem [8]. Thus, they increase the probability of finding one visible marker at the cost of more environment adaptation. Continuing with this idea, the approach in [10] uses multiple-layer markers. The main disadvantage of this approach is the accuracy waste after switching between marker layers because of scale change. Moreover, their markers require larger sizes to have similar recognition rates to that of a common single marker.

ARTag is a marker tracking system [11] that uses edge segmentation for the identification of target patterns. It is more robust than ARToolkitPlus against bad illumination conditions, and it is capable of closing broken contours [12]. Nonetheless, it only supports small occlusions, such as putting the finger on the marker.

In [13], a method that tracks the bounding box of the marker is detailed. It supports strong occlusions and does not add extra information to the marker or the environment. It is ideal for mobile devices because of its low computational cost. However, it only updates four degrees of freedom (translation in X , Y , Z axes and rotations in Z axis), so it is not a valid solution for all augmented reality applications.

The approach in [14] presents two new marker designs. Although not designed for occlusions, they support some restrictive partial occlusions, as they use the frame of the marker to codify their digital identification. An incremental feature tracking to handle marker occlusions is also described. It assumes that the marker lies on a textured plane, from which features are detected. This is not always possible, so this requirement is a scene dependent. Furthermore, according to the authors, this technique is limited to a few seconds because of drift.

In [15,16], an incremental feature tracking is also applied. In these cases, features inside the marker are detected and tracked. A correct spatial configuration of features is required to guarantee a minimum number of features and to obtain a precise homography between two consecutive frames. Thus, they are not a valid method for all markers, because they depend on the patterns that are inside the marker. They also suffer from the problem of drift.

The method in [17] uses a particle filter to obtain the six degrees of freedom of the camera when the ARToolkitPlus library fails. It minimizes the reprojection error between the marker corners and the features detected in the current image. The number of particles should be large enough in order to adapt to sudden movements. Nevertheless, the computational cost of this operation is too high. Therefore, the robustness is jeopardized by the number of particles, that is, by the real-time requirement.

The method in [18] describes a new marker design that is based on randomly scattered dots (random dot markers). Each dot is described by the spatial location of its neighbor points (LLAH geometric descriptor [19]), and a minimum set of dots must be identified to recognize the marker. Occlusions that are supported depend on the spatial distribution of dots. A uniform distribution would be ideal but penalizes distinctiveness of the geometric descriptors. The computational cost of this approach, 50–60 ms when using the publicly available implementation of the author[†] and the same hardware configuration as that presented in the experiments, exceeds real time requirement (~ 33 ms). In addition, this solution cannot be adapted to popular and existing marker tracking systems such as ARToolkitPlus.

Compared with the solution in [8], our solution only uses a single marker, so it minimizes environment adaptation. Furthermore, in contrast to the methods in [14–16], our proposal combines an incremental tracking with a tracking based on appearance to offer a robust tracking and avoid drift. Additionally, our design is based on customizable textures that are placed in the frame of the marker, which generates, by default, a uniform distribution of features, that is, it is not scene dependent. Unlike the systems in [17,18], our system is oriented to real-time applications, without losing robustness. Another advantage over the system in [18] is that our new marker design can be used directly by popular marker tracking systems such as ARToolkitPlus, that is, it is highly adaptable.

3. JUSTIFICATION OF THE NEW DESIGN

Partial occlusion of the marker causes tracking failure, because none of the detected candidates in the image

are considered as a marker because of the change in shape. ARToolkitPlus, for example, fails even when the marker is occluded approximately 2%. Furthermore, for square markers, such as those used by ARToolkitPlus, a marker occlusion generally means that the square shape of the marker is lost. Thus, the camera pose cannot be updated, because the four corners of the marker (with known 3D coordinates) are not identifiable to compute the homography that transforms the marker from its origin to the corresponding location in the current frame. However, the homography computation does not specifically need the four corners, it requires four nonlinear matches. To overcome this constraint, we should have as many features as possible with known 3D coordinates, increasing the probability of finding, at least, four 2D–3D matches during the marker occlusion.

On the basis of the statements mentioned earlier, we propose a new marker design that consists of adding customized textured borders to the marker (Figure 2, middle), which is an area not used by the ARToolkitPlus library. Furthermore, the pipeline of ARToolkitPlus remains unchanged, as the presence of these customized borders does not distort the marker identification step of ARToolkitPlus, and they offer us more information in case of occlusion. Additionally, each border is painted with a different color (Figure 2, right) to accelerate the matching step. The color codification lets us distinguish the border that each point belongs to, whereas different texture patterns guarantee that not all the points look like the same point. It is also noteworthy that the borders of the marker have been slightly expanded (the new marker size is 12% larger) to obtain reasonable texture sizes and to maintain a thin black outer border. We have observed that this black outer border benefits the marker detection of the ARToolkitPlus pipeline [9], because the square shape is more easily detected after the thresholding step despite the presence of the textures. Likewise, we have tried to not increase too much the global size of the marker in order to be a valid solution for most applications, especially those that have space limitations.

In short, the novelty of our design is that we exploit an area of the marker that is not being used to incorporate some texture patches that provide us more information in case of occlusion and let us calculate the new camera pose (more robustness). In addition, the presence of these textures do not jeopardize the execution of existing marker tracking systems (high adaptability), and their appearance can be customized by the users, which is an interesting property for marketing purposes.

It is noteworthy that our design can also be used with nonsquare markers, such as those that have a circular shape. Our proposal uses the texture patches that are provided in the training phase as a reference, taking into account where they are placed. Hence, only the placement of the textures should be adapted to make our design feasible to other marker shapes.

[†]<http://hvrl.ics.keio.ac.jp/uchiyama/me/code/UCHIYAMARKERS/index.html>

4. OCCLUSION PIPELINE

The proposed marker tracking system is composed of two main modules: the ARToolKitPlus pipeline and the Occlusion pipeline. The first one is the common ARToolKitPlus pipeline detailed in [9], whereas the second one is the solution presented here to update the camera pose against the partial occlusions of the marker. The interaction between these two modules is shown in Figure 3. When the ARToolkitPlus pipeline is executed successfully, the datum associated to the current state is stored (*Update Occlusion Data*). This datum is used to initialize the Occlusion pipeline when the ARToolkitPlus pipeline fails; thus, it guarantees that the occlusion state is updated every time the marker is visible.

Furthermore, the Occlusion pipeline combines two different tracking methods: *frame-to-frame tracking* and *tracking by detection*. The first one is based on temporal coherence, which provides a robust tracking against no-strong camera movements and has a low computational cost. However, when a strong camera movement

is applied, the frame-to-frame tracking fails, so a second tracking method that computes the pose despite the movement applied to the camera has been incorporated to the Occlusion pipeline. This second method, named tracking by detection, has higher computational cost because it recognizes the presence of the textures in the image by using their appearance, without temporal coherence assumptions. Therefore, the main idea of combining both methods is based on using the frame-to-frame tracking as long as possible because of its robustness against smooth camera movements and low computational cost, and using the tracking by detection only when the frame-to-frame tracking fails, as a recovery mode, because it requires more computational cost. Likewise, the tracking by detection allows less degree of occlusion, because the matching quality between the image appearance and the reference appearance should be reasonably high to not generate false positives.

The novelty of the proposed tracking system (Occlusion pipeline) is how two different tracking methods are combined to exploit the advantages of each approach, obtaining a robust and real-time tracking. Nonetheless, although these tracking methods are based on existing solutions, we also offer some novel ideas. In contrast to the methods in [14,15], which use a similar incremental tracking of points, our frame-to-frame tracking incorporates a refinement step, which minimizes the problem of drift (Section 5). The tracking by detection, meanwhile, combines an efficient implementation of the Scale Invariant Feature Transform (SIFT) descriptor [1] with a color identification step, which simplifies the complexity of the recognition (Section 4.2.2). Additionally, our tracking by detection exploits the properties of the new marker design to perform an intelligent search of the homography that best represents the camera pose, that is, each point that is used to generate a homography hypothesis is selected from a different texture to avoid the selection of collinear points that produces a singularity.

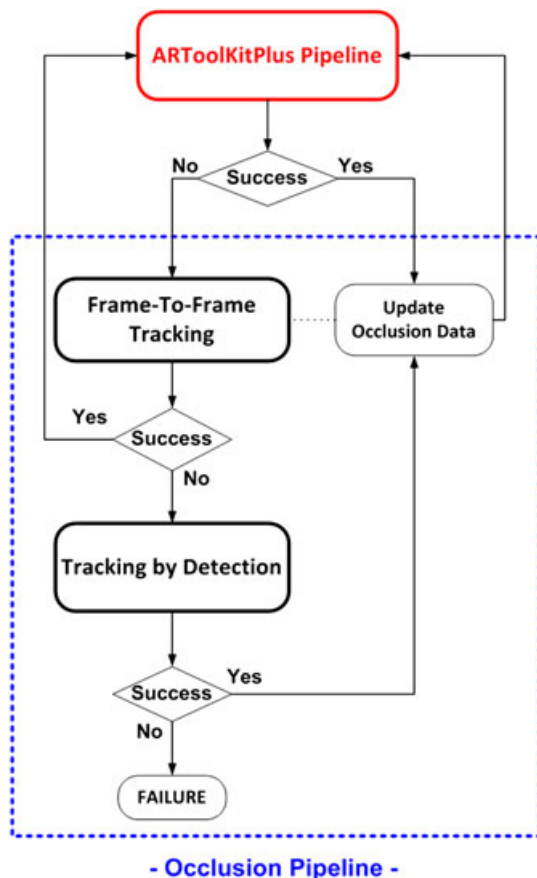


Figure 3. Overview of the Occlusion pipeline. It shows how the ARToolkitPlus and Occlusion pipelines interact, as well as how the two different tracking methods (*frame-to-frame tracking* and *tracking by detection*) are mixed.

4.1. Offline Phase

The tracking by detection method is based on appearance, so it requires a database of feature descriptors that belong to the marker, which is computed during an offline phase (Figure 4). For that purpose, it is necessary to train the marker along a set of keyframes. These keyframes are processed, and the 2D features that belong to the surface of the texture borders are back-projected, obtaining their corresponding 3D values. In addition, these 3D points and their corresponding descriptors are indexed in a database, which is used to find a set of matches during the online phase. This training phase is executed only once for each marker design. All the steps involved in this phase are detailed as follows.

4.1.1. Keyframe Selection.

The number of keyframes depends on the set of movements that the camera covers at runtime, without exceeding

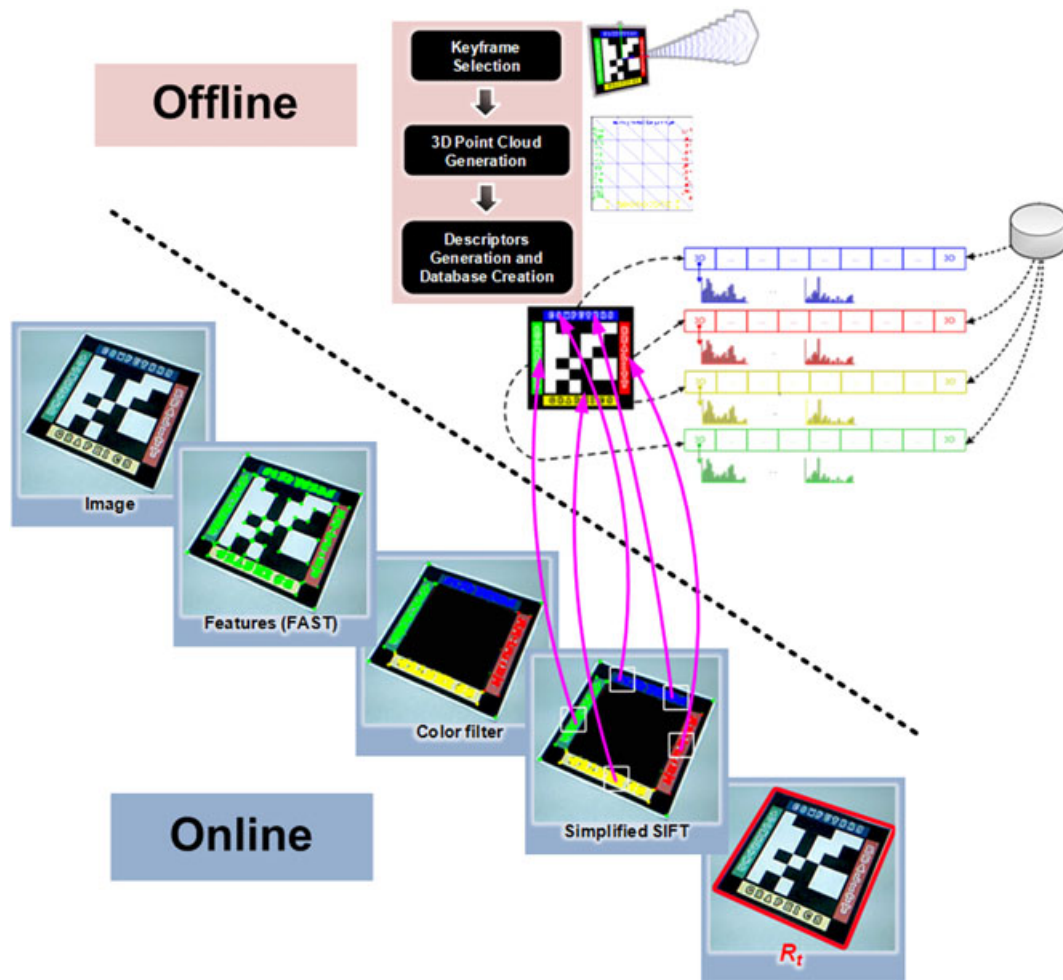


Figure 4. Overview of the *tracking by detection*.

the storage of the database. Thus, as it is carried out in [20], the marker is trained along multiple scales to solve the scale ambiguity introduced by our feature. Because of real-time requirements, we use the Feature from Accelerated Segment Test (FAST) operator [21] to extract features during the online stage, which does not provide scale information. Additionally, it is noteworthy that we only consider the keyframes that put the marker in front of the camera, without applying rotations. This is a valid assumption for an object that lies on a plane, and it has been successfully applied by other authors [2].

4.1.2. 3D Point Cloud Generation.

For each keyframe, 2D features are detected using FAST, and those that do not belong to one of the four marker borders are discarded. Additionally, the remaining 2D features are divided into four different clusters, according to the border they belong to.

Each of the stored 2D features are back-projected to obtain their 3D values. This is a necessary step because

we require a set of 2D–3D matches at runtime to calculate the camera pose. This way, the marker is modeled as a triangle mesh, and each triangle is rendered with a unique color by using the camera pose of the corresponding keyframe, which is known because it is virtually generated. Likewise, the location of each 2D feature points to a unique color, which is used to index the corresponding facet. Once each 2D feature is associated with its corresponding 3D triangle, its 3D values are calculated using barycentric coordinates [22].

4.1.3. Descriptors Generation and Database Creation.

This step computes the descriptor of each 3D point that belongs to one of four texture borders of the marker. We have chosen the SIFT descriptor [23] to that end, because it has demonstrated high performance compared with other local descriptors [24]. However, the original Difference of Gaussians feature detector has been replaced by the FAST detector. As the Difference of Gaussians is computationally

expensive for real-time applications, the FAST detector has been chosen for the online phase because of its compromise between robustness and computational cost. Nevertheless, this implies that no scale information is obtained during the feature extraction step. Therefore, to avoid scale ambiguity and obtain better matching results at runtime, we have implemented scale-space like [20]. Each 3D point is associated to multiple descriptors, one for each image that results from applying different Gaussian filters to the corresponding keyframes. Particularly, we have obtained a good compromise between storage cost and computational cost by using eight Gaussian filters, with a standard deviation factor of $1/\sqrt{2}$ the previous one (recursively). This procedure tries to cover the expected scale ranges at runtime.

The original SIFT descriptor has also been replaced with a simplified version of it [1] to reduce the computational time. These simplifications involve a fixed patch size, retaining only the most dominant orientation for each feature or the use of parallel techniques (OpenMP). Furthermore, some calculations are precomputed, such as all possible kernel orientations. As reported in [1], these simplifications let us obtain hundreds of SIFT descriptors in a few milliseconds.

Once all descriptors of all 3D points have been computed, they are indexed in a database using kd-trees. Furthermore, a different database is created for each marker border, resulting in four different databases, which offer faster and more robust matches during the online phase, because an input feature is only matched against its corresponding border database.

4.2. Online Phase

This phase is executed every time the ARToolkitPlus pipeline is not able to detect the marker. It uses the data provided by the last successful execution of the ARToolkitPlus pipeline to initialize the frame-to-frame tracking (Figure 3). This is a fast method that works fine while no fast camera movements are applied. Therefore, in case a rapid camera movement is performed and the

tracking is lost, the tracking by detection method is called, which uses the databases of descriptors created in the offline phase (Section 4.1). This second method is based on appearance and can support fast camera movements, so it is used as a recovery mode. This way, if the tracking by detection is able to compute the camera pose, the tracking method returns to the frame-to-frame tracking initialization step, starting the whole process over again. If after a few attempts the marker is not detected (*Failure* status of Figure 3), the Occlusion pipeline does not update the camera pose, and the system waits until the ARToolkitPlus pipeline is successfully executed. It is noteworthy that the ARToolkitPlus pipeline is executed every time to check the marker visibility; thus, the Occlusion pipeline is disabled as soon as the ARToolkitPlus detects the marker.

4.2.1. Frame-to-Frame Tracking.

The first step of the frame-to-frame tracking is the initialization of the data that will be used for tracking. To that end, the last successful camera pose and frame that were registered are used to back-project the features that lie on the surface of the marker and compute their representative descriptors (Figure 5). The FAST detector is used to extract these features, while the camera pose is provided by the ARToolkitPlus pipeline or by the tracking by detection method after recovering from a failure. The back-projection process is similar to that explained in previous sections, using barycentric coordinates to compute the 3D values. Nevertheless, these back-projected features and descriptors are not the same as those generated in the offline phase. These features belong to the entire surface of the marker, that is, they are not limited to the frame of the marker. Furthermore, they do not use the robust SIFT descriptor but a simple descriptor based on the grey values of their surrounding patches (we will refer it as GREY descriptor). It is similar to normalized correlation, but using a Gaussian weight.

$$GREY(p) = \frac{1}{N} * \sum_{s \in S} I(p+s) * G(s) \quad (1)$$

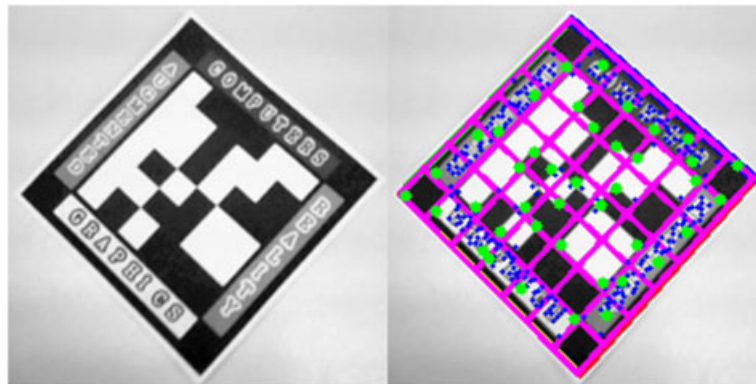


Figure 5. Frame-to-frame tracking initialization. Image of the last successful ARToolkitPlus execution (left). Feature detection (blue points) and back-projection of the strongest features (green points) that are uniformly distributed (right).

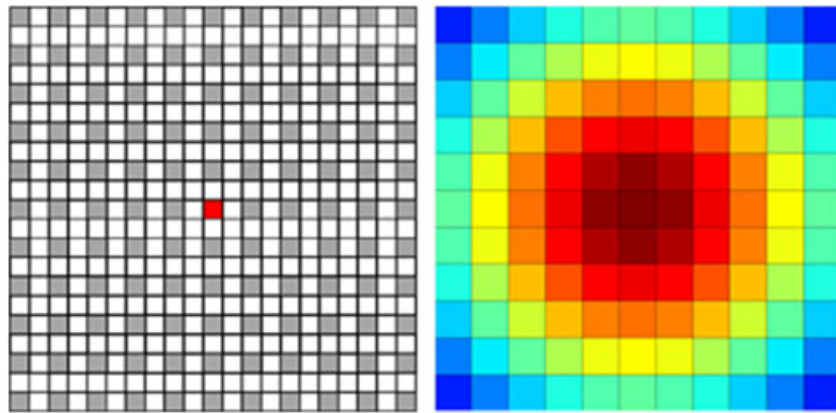


Figure 6. Sparse 11×11 sampling grid (left) and the Gaussian weight (right) used by the GREY descriptor. Red color indicates samples with more weight.

where $N = \|\sum_{s \in S} I(p+s) * G(s)\|_2$, S is the set of samples that are considered around the point p , I represents the intensity of the image, and G is a Gaussian weight. More precisely, a sparse 11×11 sample grid centered on each feature is used to build a 121-bin descriptor (Figure 6). This descriptor is not as robust as SIFT, but it has demonstrated considerable robustness and low computational cost (Section 5).

Because of the reduction of the computational cost, the maximum number of back-projected features has also been limited to 49. The marker has been divided into 49 equal squares (7×7 grid), and only the strongest feature is retained for each square. This is a fast method to limit the maximum number of features and maintain a uniform spatial distribution of features, which favors the response against occlusions, because it does not matter which side of the marker is occluded. This initialization step is executed every time the marker changes from visible to occluded, as it guarantees that all features and descriptors are updated with the last visible frame; thus, it is adapted to possible environmental changes, such as different illumination conditions.

Once the initialization step has been executed, all back-projected features that are visible in the image are tracked in the successive frames by using an incremental tracking similar to those presented in [14,15]. First, the new feature locations are predicted using a pyramidal optical flow [25]. This technique provides a good starting point for the second step, which searches in the vicinity of predicted locations the presence of points with similar GREY descriptors (Figure 7). This way, a feature is considered successfully tracked if a point is found in a 20×20 window that is centered on the predicted location and whose descriptor is highly correlated to the original descriptor (correlation value higher than 0.9). This step removes outlier matches and minimizes the drift problem that this type of techniques typically suffers. For all features that pass this rule, their location prediction is replaced by the location of the point they are matched to. Nonetheless,

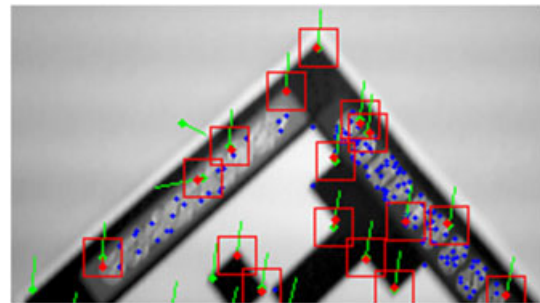


Figure 7. Snapshot of the *frame-to-frame tracking*. Green lines indicate optical flow prediction, whereas the red dots are the final location after the refinement step. The red squares represent the area of each feature that is used for the refinement step. Features without the red square are considered as outliers.

some of these remaining matches can be outliers, so we use RANSAC [26] to compute the affine transformation that represents the movement of the inlier features between the two consecutive frames. It is by using this affine transformation that the new 2D locations of the 3D features computed in the initialization step, which are used to update the camera pose, are known. We require a minimum set of inlier matches (six in our experiments) to validate the correctness of the current tracking. Finally, the 2D location of the 3D features are updated for the next frame. These 3D features are projected with the new camera pose, and their 2D position is refined using a refinement step similar to that mentioned earlier. If there is a point with similar GREY descriptor in the 10×10 vicinity of the feature projection location, then its position is changed, and its descriptor is updated. Otherwise, both the position and the descriptor are not changed, although they are considered for the next frame as well. This last refinement step ensures that the descriptors evolve, and it improves the quality of the optical flow for the next frame. The simple projection of a 3D point by using a nonaccurate camera pose may fall

in the middle of a homogeneous image area, which is not an optimal scenario for optical flow techniques. We also require that, at least, 15% of visible features are updated using this refinement procedure to validate the correctness of the current tracking. This requirement, together with the affine restriction, identifies bad tracking frames due to fast movements or drift, and helps us to decide when to execute the tracking by detection.

4.2.2. Tracking by Detection.

This phase serves for recovering the camera pose without any user intervention in case the spatial coherence fails.

Each of the four borders of the marker has been codified with a unique color, building a different database for each one (Figure 4). Thus, for each feature detected in the current image, the border it belongs to is calculated. To do this, for each detected feature, we count the number of pixels of its vicinity (10×10 window) that belong to each color. To determine the color of each pixel, its corresponding Hue-Saturation-Value (HSV) values are considered. If the saturation or value components of a pixel are in the extremes of their range, then this pixel is not considered as colored but rather as a white or black pixel. Otherwise, the hue channel specifies the color of a pixel. Moreover, to consider a feature as colored, 50% of their surrounding pixels must belong to the same color; that is, there must be a dominant color. This color codification step is very useful, because it reduces the number of matching combinations by 75% for each feature and rules out too many points that do not belong to the frame of the marker.

Once the current detected features that do not belong to the frame of the marker have been discarded, the simplified SIFT descriptor is calculated for the remaining detected features. Furthermore, each of these features has been assigned to one of the four borders of the marker; thus, only the corresponding database is used to have a correspondence. The query to the database returns the two closest descriptors (d_1 and d_2 , respectively) according to the input descriptor (d_i) and considering that the Euclidean distance is used to determine the proximity between two descriptors. This way, we consider that d_1 and d_i are positively matched if $\text{dist}(d_1, d_i) < k * \text{dist}(d_2, d_i)$, where $k \in [0..1]$ is a control parameter that discards matches with poor distinctiveness.

As a result of matching current detected features with those stored in the database, we obtain a set of 2D–3D matches. However, some of these matches may be outliers, so we compute the camera pose by using the robust hypothesize-and-verify PROSAC method [27]. Additionally, we have designed a new sample selection criterion that favors the quality of the homography that represents the transformation between the two sets of points. First, we choose randomly two borders of the marker that have, at least, two correspondences. Then, two matches for each of those two borders are selected according to their matching quality, which is inversely proportional to the distance between its corresponding descriptors. This criterion

overcomes the restriction of the homography generation, which states that all the points cannot be collinear. In addition, we have observed that this criterion returns more stable homography transformations because of the better distribution of selected points. The maximum number of matches has also been limited with the aim of running the application in real time. This way, the matches with the highest quality will be selected first.

5. EXPERIMENTS

In this section, we investigate several characteristics of the new marker design that is proposed. The hardware setup consists of an Intel Core 2 Duo processor (Intel: Santa Clara, California, USA) at 2.40 GHz and 2 GB of RAM equipped with a Logitech QuickCam Connect (Logitech: Morges, Switzerland) webcam. All the experiments have been executed with a Windows 7 (Microsoft: Redmond, Washington, USA) operating system.

5.1. Tracking by Detection

First, the response of the tracking by detection method has been studied. Multiple marker designs have been used (Figure 8) to investigate the importance of choosing good texture content. More precisely, four different marker designs have been developed to simulate multiple conditions: *Words* (Figure 8(a)) represents a typical design that can be used for marketing purposes, encoding some words of a message along the frame of the marker; *Random-characters* (Figure 8(b)) is similar to *Words* but uses randomly selected characters to demonstrate that no specific letters are required; *Random-shapes* (Figure 8(c)) symbolizes the textures that are made using logos; and *Repetitive-symbols* (Figure 8(d)) is referred to those shapes that have a repetitive pattern and do not provide distinctiveness.

The scale is the first parameter that has been examined. Nevertheless, this parameter is not dependent on the marker design but on the size of the textures. This way, after executing tracking by detection with different marker designs at multiple distances, we can conclude that the marker width must be 20% of the image width to obtain a good detection. In addition, we observed that no false positives were detected during the experiments, that is, the presence of the marker was not detected unless it was actually there. This is a desirable property, as this is the starting point for the frame-to-frame tracking method that is based on temporal coherence.

To obtain these results, as well as those presented next, a set of images with a known camera pose are required to decide the success of the detection. According to this, a set of snapshots was acquired in which the marker is completely visible. These images have been processed with the normal ARToolkitPlus pipeline, obtaining the corresponding camera pose for each one. Moreover, we have used these poses to draw black patches on the surface of the marker, simulating marker occlusions (Figure 9).

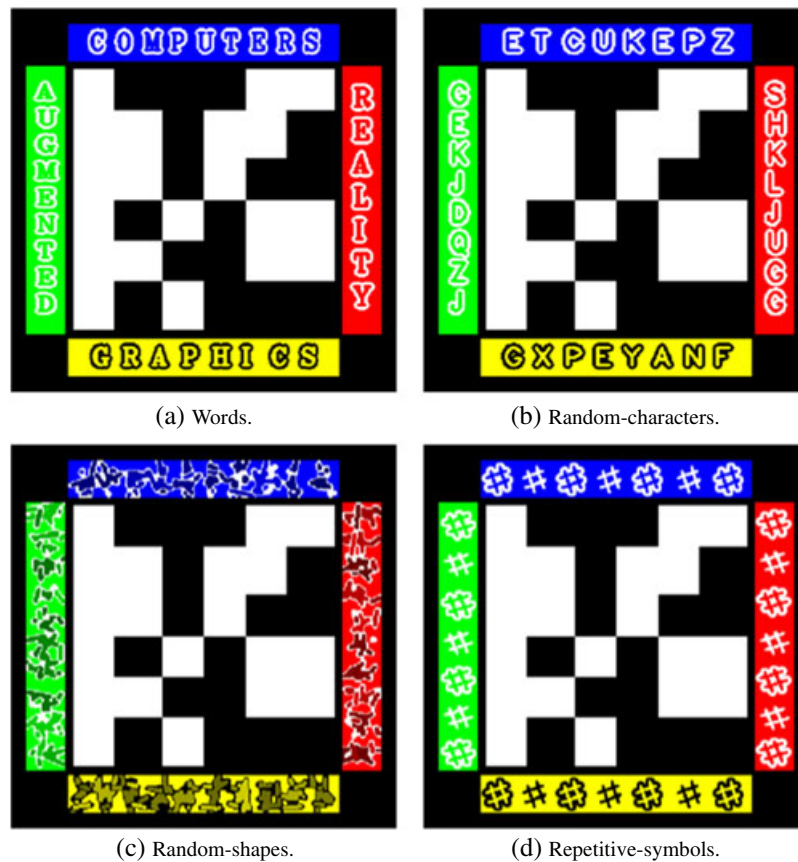


Figure 8. Different marker designs that have been used to investigate the importance of choosing a good texture content.



Figure 9. Three image examples that have been used in our experiments to simulate the partial occlusion of the marker.

Using the procedure described earlier, together with the scale ranges in which the tracking by detection method is executed successfully, we have taken a set of snapshots for each marker design of Figure 8. These snapshots have been used to perform several executions of the same image set but with a different SIFT parametrization. Figure 10 shows the success ratio of the tracking by detection method for each marker design, according to the SIFT parametrization and for different image resolutions. We can observe that the larger the SIFT descriptor patch is, the higher the ratio of success is obtained, because more stable descriptors are built. Nonetheless, the computational cost increases

proportionally with the size of the patch, so we have to reach a compromise between robustness and computational cost. Furthermore, the image resolution also influences this decision, because a patch of fixed size covers different amounts of image at different resolutions. Therefore, at lower resolutions, a smaller patch size is required to obtain results similar to those achieved at higher resolutions. The other two parameters of the SIFT descriptors control the number of regions that divide each patch and the size of the histograms that characterizes the gradients of each region [23]. These values are related to the size of the descriptor, and they offer us the possibility of establishing the degree

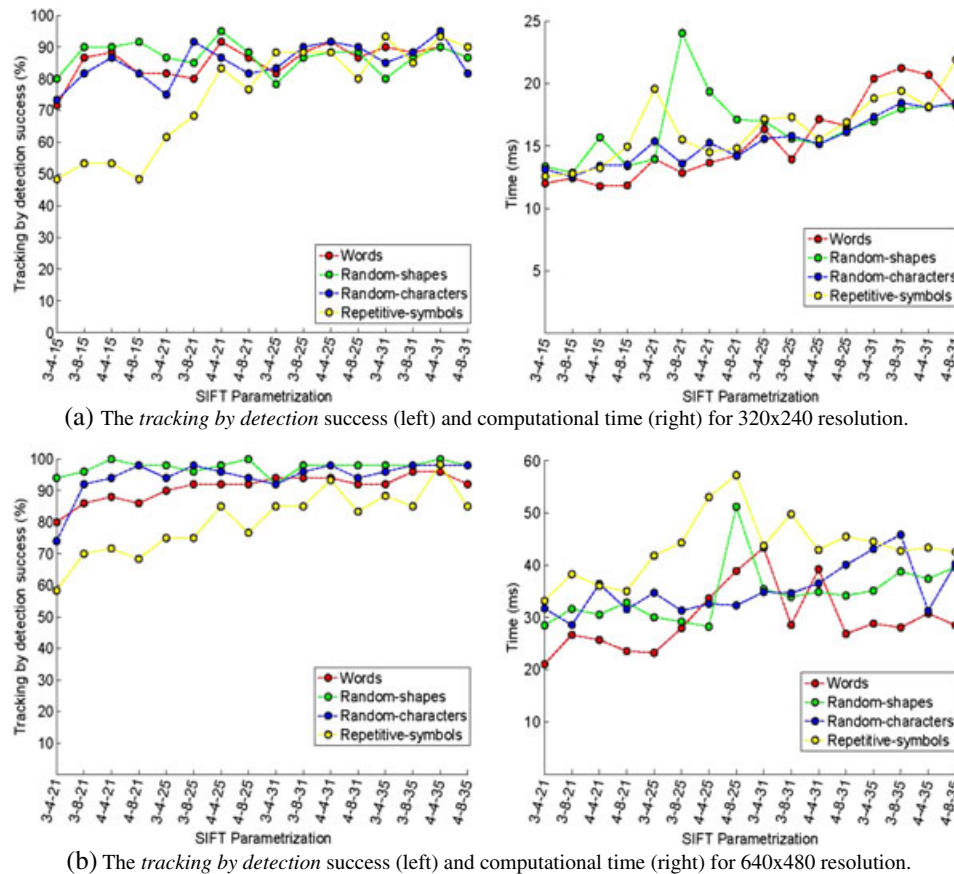


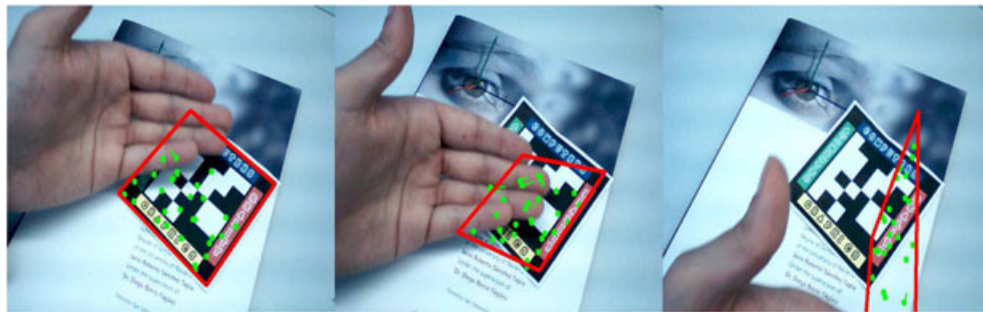
Figure 10. The *tracking by detection* success and computational time according to the SIFT parametrization and image resolution. The i - j - k parametrization of SIFT (x-axis) corresponds to the number of regions (i), the number of histogram bins (j), and the patch size (k) that is used to build the SIFT descriptor [23].

of descriptor distinctiveness that best suits the properties of the corresponding textures. According to our experiments (Figure 10), the best results are obtained when the patch is divided into 4×4 regions, whereas the number of histogram bins that codify each region should be between 4 and 8. Additionally, although a different set of images has been used for each marker design, which could explain small differences in the success ratio between two different marker designs, analyzing the results of Figure 10 for 640×480 resolution, we can establish that the textures with a repetitive pattern (*Repetitive-symbols*) harm the descriptor distinctiveness. The good response of *Random-characters* and *Random-shapes* designs demonstrates that textures with words and shapes are valid solutions, because they build good quality descriptors as a result of generating many unique patches around each 3D point. The response of *Words* proves that positive detection results are also obtained using textures with customized words, as long as a *repetitive pattern* does not appear. Taking everything into consideration, we propose the 4-4-25 and 4-4-31 SIFT parametrizations for 320×240 and 640×480 resolutions, respectively, as they are the options that generally provide

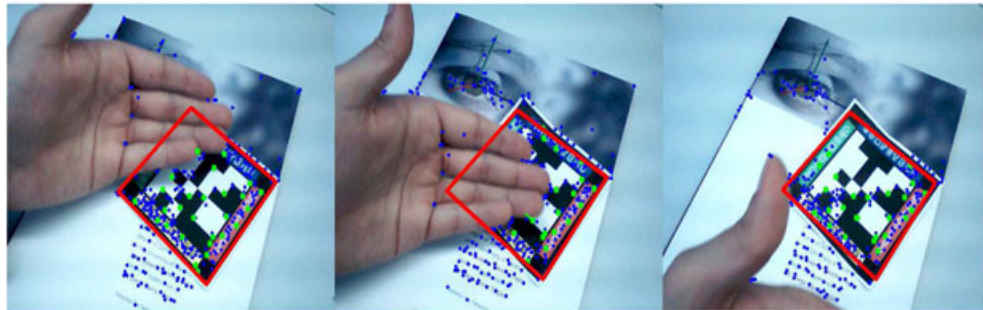
the best compromise between the success ratio and the computational cost.

5.2. Frame-to-Frame Tracking

The quality of the frame-to-frame tracking has also been studied. However, we have only executed the experiment for the *Words* marker design, as the results for the other designs are similar. The same video sequence has been executed twice, one without the refinement step of the frame-to-frame tracking and the other one using the refinement process. It should be noted that the incremental tracking of points that is described in [14,15] is very similar to our limited version of frame-to-frame tracking, which disables the refinement step. Thus, the main objective of this experiment is to compare our frame-to-frame tracking with a simulated version of [14,15]. Although the video is at 640×480 resolution, the frame-to-frame tracking is executed at 320×240 resolution to reduce the computational cost. In the video, the marker is occluded by the user's hand movement, and the marker lies on a textured

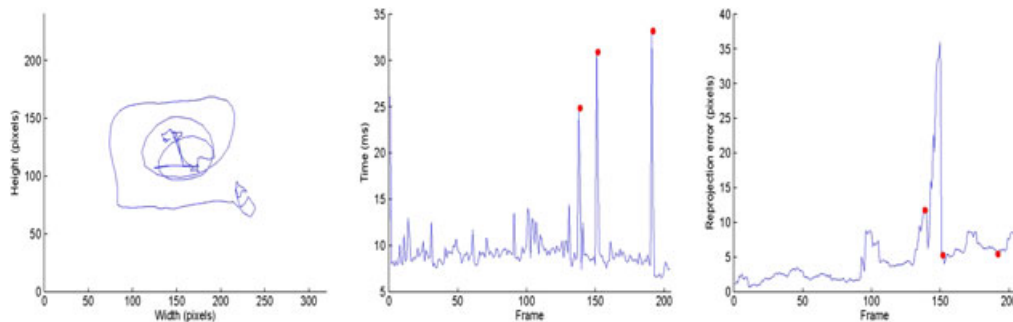


(a) Frame to frame tracking without refinement step.

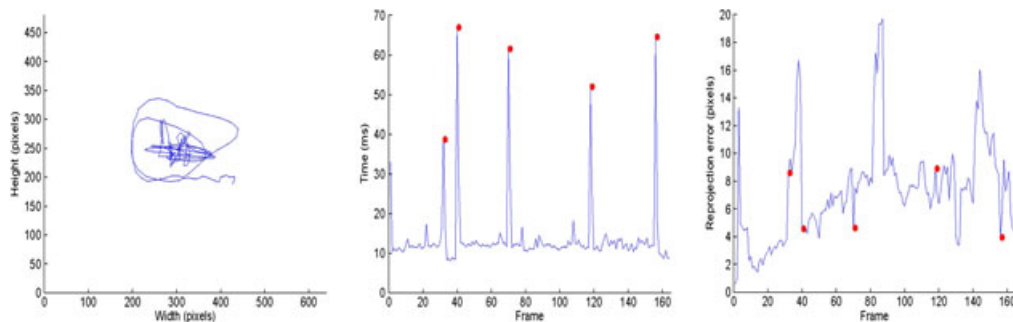


(b) Frame to frame tracking with refinement step.

Figure 11. Camera pose (red square) obtained (a) using or (b) not using the refinement step of the *frame-to-frame tracking*. These two image sequences correspond to the same three frames of an input video. See text for more details.



(a) Marker movement (left), computational time (middle) and reprojection error (right) for the execution of Occlusion pipeline at 320x240 resolution.



(b) Marker movement (left), computational time (middle) and reprojection error (right) for the execution of Occlusion pipeline at 640x480 resolution.

Figure 12. The response of the Occlusion pipeline for a video sequence (performance and error). The image position of the center of the marker throughout the entire sequence is shown on the left. Both tracking methods are used during the execution. Red dots indicate the execution of the *tracking by detection*. The computational time for these frames is the sum of both tracking methods.

surface to increase the difficulty of tracking. As shown in Figure 11, this experiment demonstrates the robustness of the refinement step, which returns a good pose despite the hand occlusion, because poorly tracked features are corrected using the correlation of the GREY descriptors. The absence of the refinement step causes an increase in tracking error because of the poor prediction of the optical flow, which is confused by the hand movement. The computational time for the two executions using a nonoptimized code was 8.1 and 13.05 ms, respectively. The time difference is due to the cost of computing and matching

the GREY descriptors that are necessary for the refinement step. From this observations, we could say that the frame-to-frame tracking that is proposed here offers higher robustness compared with other approaches [14,15] at the cost of a little computational cost. Nevertheless, we have to point out that this is the worst scenario for the frame-to-frame tracking, especially for the version that uses the refinement step, because there are many visible features that must be tracked (none of the marker points fall outside the image), and there are many points around the marker (blue points of Figure 11(b)), whose GREY descriptors

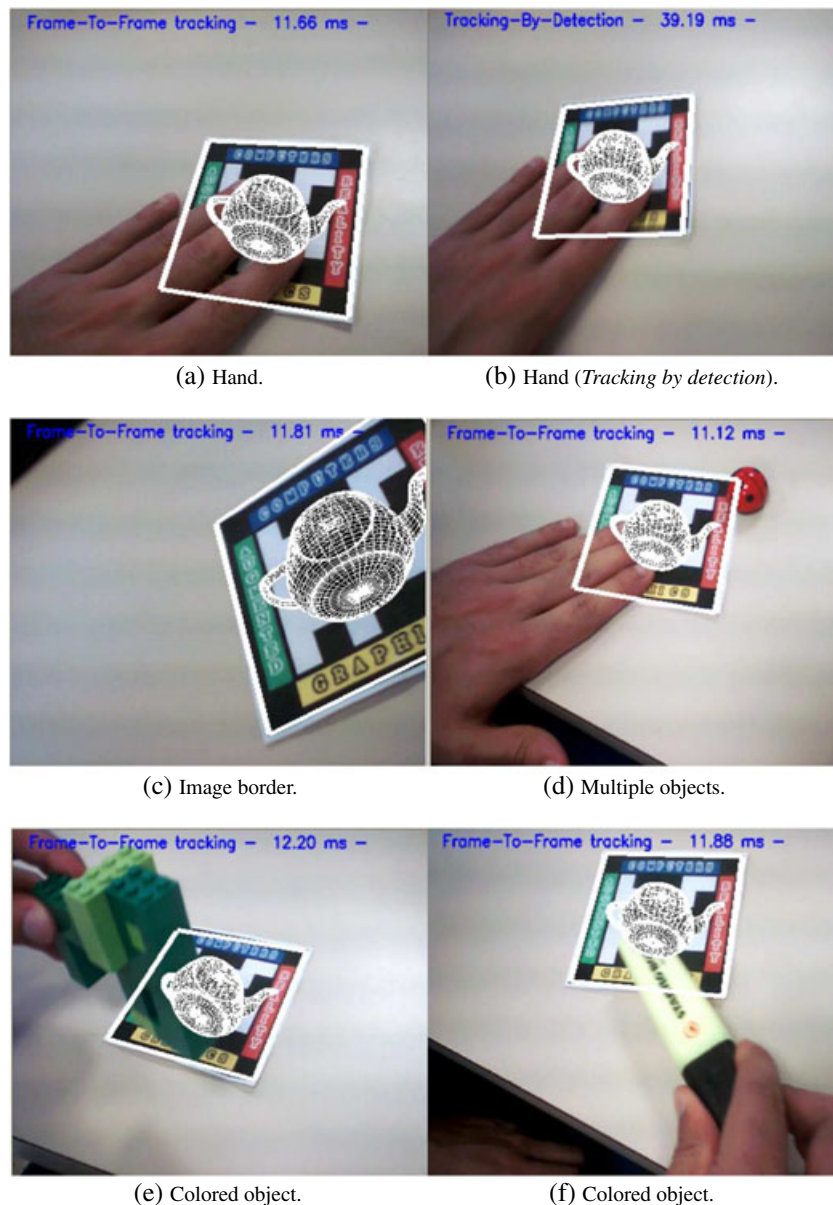


Figure 13. The output of the Occlusion pipeline for different types of marker occlusions. A virtual teapot, as well as the marker bounding box are rendered to indicate success.

must be computed and which can produce false matches. Despite all this, the frame-to-frame tracking runs in real time and is able to compute a good camera pose.

5.3. Occlusion Pipeline

Figure 12 shows the response of our Occlusion pipeline for a video sequence where both tracking methods are mixed. To have the correct pose of each frame of the video and compute the reprojection error (Figure 12, right), the marker was visible during the entire sequence in order to successfully execute ARToolkitPlus. Moreover, we have used the procedure described earlier to simulate marker occlusions in each frame of the video. The image position of the center of the marker along the entire sequence is shown on the left of Figure 12 to indicate that the marker is continuously moving and even making fast movements that impede tracking. The red dots of Figure 12 represent the execution of the tracking by detection. Thus, the



Figure 14. An example of occlusion signal output.

frame-to-frame tracking is executed as long as possible because of its low computational time (Figure 12, middle). When this method fails as a consequence of a fast movement or error accumulation, the tracking by detection is called to compute the correct camera pose and restart the frame-to-frame tracking. This way, the reprojection error between the correct camera pose and that returned by the Occlusion pipeline (Figure 12, right) is maintained at reasonable levels most of the time. The few frames that have a higher reprojection error are those frames that the frame-to-frame tracking takes to indicate failure. Nevertheless, the time it takes to react is very small (with nonoptimized code), and the results are still visually acceptable. After executing this experiment with two different image resolutions (320×240 and 640×480), we can say that the results are similar. At 320×240 resolution, we obtain lower computational cost at the expense of higher reprojection error, whereas at 640×480 resolution, we obtain less reprojection error at the expense of more computational cost.

Finally, in Figure 13, the output of the Occlusion pipeline for different types of marker occlusions is presented. Figure 13(a) demonstrates that our Occlusion module supports hand occlusions, including recovery from a fast camera movement (Figure 13(b)). Figure 13(c) shows the ability to compute the camera pose when the marker is partially outside the image. In addition, Figure 13(d) represents the tracking success when the marker is occluded by multiple objects. Likewise, Figure 13(e,f) displays the robustness against occlusions made by objects with colors similar to the borders of the occluding object.

5.4. Exploring Interaction Possibilities

The new marker design that is proposed provides us more information when the marker is occluded, which can be used to develop human-machine interfaces. These interaction possibilities are complementary to those interfaces created by other authors, who have also used occlusions to implement them. For example, the interface in [28] places multiple markers at specified locations, and depending on

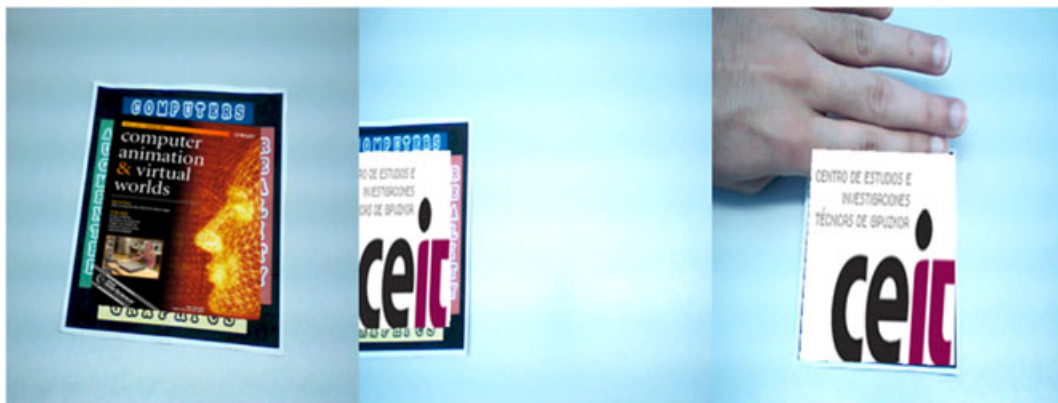


Figure 15. Photo viewer output (left). Transition to the previous photo due to the left border occlusion (middle), and the positive zoom applied because of the up border occlusion (right).

which markers are occluded and visible, it interprets different user actions. In a similar way, the interface in [29] presents a simple hand gesture recognition using the interface in [15] to threat small finger occlusions. The interface in [18], meanwhile, detects which region of the marker is occluded rectifying the viewpoint of the marker in the image and subtracting the rectified marker and the reference. It is noteworthy that all these interfaces can be adapted to our design, although the interaction possibilities that we show here are more difficult to obtain or computationally more expensive if other designs are used.

All the interaction possibilities presented here have to determine which features of the surface of the marker are visible. This is a simple task when the marker is not completely inside the field of view of the camera, because the projection of the occluded points fall outside the image. However, this is not the case for occlusions caused by other objects, such as the user's hands. For these cases, only those features of the surface that have a point with similar GREY descriptors in its vicinity are considered as visible. This visibility test has a negligible computational overhead for our solution because this information is already generated by our frame-to-frame tracking, that is, our design offers the ability to efficiently develop human-machine interfaces. Two interaction possibilities are presented as follows as a demonstration, but it can be extrapolated to numerous applications.

- Occlusion signal

The textures that have been placed along the frame of the marker imply the detection of multiple features in each border. Using this assumption, we can deduce the degree of occlusion depending on the number of occluded points. We can display an occlusion signal (Figure 14) informing the user about the degree of occlusion. This is similar to the idea of a coverage signal used by mobile devices.

- Photo viewer

Because of the presence of textures, in each border of the marker, there are multiple features. Thus, we can detect which side of the marker is occluded and identify the border with fewer visible features. Moreover, the occlusion of each border can be interpreted as a different action. To prove this usability, we have implemented a photo viewer (Figure 15), for which a different action is executed depending on which border is occluded:

- left border: go to the previous photo.
- right border: go to the next photo.
- up border: positive zoom to the current photo.
- down border: negative zoom to the current photo.

6. CONCLUSIONS

A new marker design for the treatment of occlusions has been described. It is based on customizable textures that are placed along the frame of the marker, which provide extra information during the occlusion. This way, our design can

be adapted to any marker that does not codify information on its frame, making it highly adaptable. Moreover, the ability to customize textures lets users design their own markers or expand the marketing opportunities of the companies, which is a desirable property. Some guidelines for the creation of these textures, which favors robustness during occlusion, have been detailed.

To exploit the extra information provided by the presence of textures, two different tracking methods have been proposed to compute the camera pose. The first one is a fast method based on temporal coherence, which works fine while no fast camera movements take place (frame-to-frame tracking). If this method fails, another tracking method is called (tracking by detection), which is computationally more expensive but supports rapid camera movements. Experimental results illustrate the robustness of the combination of these techniques, offering a real-time solution (with nonoptimized code). Indeed, the proposed tracking outperforms other solutions, because it does not suffer from drift. An in-depth study of the parametrization of these tracking methods has also been developed to show their response against different configurations and to obtain the best balance between robustness and performance.

In the future, we plan to apply this work in mobile platforms. We will study the optimizations that are required to achieve robustness and real time in these devices, which have limited capabilities.

ACKNOWLEDGEMENTS

The contract of Hugo Álvarez is funded by the Government of the Basque Country within the *Formación de Personal Investigador* program that belongs to the *Education, University and Research* department. We are thankful to the *Beca Universidad de Navarra* of the University of Navarra for funding the contract of Ibai Leizea.

REFERENCES

1. Sánchez JR, Álvarez H, Borro D. Towards real time 3D tracking and reconstruction on a GPU using Monte Carlo simulations, In *9th IEEE International Symposium on Mixed and Augmented Reality (ISMAR'10)*, Seoul, Korea, 2010; 185–192.
2. Xu K, Chia KW, Cheok AD. Real-time camera tracking for marker-less and unprepared augmented reality environments. *Image Vision Computing* 2008; **26**(5): 673–689.
3. Pressigout M, Marchand E. Real-time 3D model-based tracking: combining edge and texture information, In *IEEE International Conference on Robotics and Automation (ICRA'06)*, Orlando, Florida, USA, 2006; 2726–2731.
4. Vacchetti L, Lepetit V, Fua P. Combining edge and texture information for real-time accurate 3D camera

- tracking. In *Proceedings of the 3rd IEEE/ACM International Symposium on Mixed and Augmented Reality (ISMAR'04)*, Arlington, VA, USA, 2004; 48–57.
5. López de Ipiña D, Mendonça PRS, Hopper A. TRIP: a low-cost vision-based location system for ubiquitous computing. *Personal and Ubiquitous Computing* 2002; **6**: 206–219.
6. Mohring M. Video see-through AR on consumer cell-phones. In *Third IEEE and ACM International Symposium on Mixed and Augmented Reality (ISMAR'04)*, Arlington, VA, USA, 2004; 252–253.
7. Zhang X, Fronz S, Navab N. Visual marker detection and decoding in ar systems: a comparative study. In *Proceedings of the 1st International Symposium on Mixed and Augmented Reality (ISMAR'02)*, Darmstadt, Germany, 2002; 97.
8. Kato H, Billinghurst M. Marker tracking and HMD calibration for a video-based augmented reality conferencing system. In *Proceedings of the 2nd IEEE and ACM International Workshop on Augmented Reality (IWAR'99)*, San Francisco, California, USA, 1999; 85–94.
9. Wagner D, Schmalstieg D. ARToolkitPlus for pose tracking on mobile devices. In *Proceedings of 12th Computer Vision Winter Workshop (CVWW'07)*, St. Lambrecht, Austria, 2007; 139–146.
10. Tateno K. A nested marker for augmented reality. In *IEEE Virtual Reality Conference (VR'07)*, Charlotte, North Carolina, USA, 2007; 259–262.
11. Fiala M. ARTag, a fiducial marker system using digital techniques. In *Proceedings of the IEEE Computer Society Conference on Computer Vision Pattern Recognition (CVPR'05)*, San Diego, California, USA, 2005; 590–596.
12. Fiala M. Comparing ARTag and ARToolkit Plus fiducial marker systems. In *IEEE International Workshop on Haptic Audio Visual Environments and their Applications (HAVE'05)*, Ottawa, Ontario, Canada, 2005; 148–153.
13. Álvarez H, Borro D. A novel approach to achieve robustness against marker occlusion. In *International Conference on Computer Vision Theory and Applications (VISAPP'09)*, Lisboa, Portugal, 2009; 478–483.
14. Wagner D, Langlotz T, Schmalstieg D. Robust and unobtrusive marker tracking on mobile phones. In *IEEE International Symposium on Mixed and Augmented Reality (ISMAR'08)*, Cambridge, UK, 2008; 121–124.
15. Malik S, Roth G, McDonald C. Robust 2D tracking for real-time augmented reality. In *Proceedings Conference Vision Interface (VI'02)*, Calgary, Canada, 2002; 399–406.
16. Maidi M, Ababsa F, Mallem M. Handling occlusions for robust augmented reality systems. *Journal on Image and Video Processing* 2010; **2010**: 1–12.
17. Marimon D, Maret Y, Abdeljaoued Y, Ebrahimi T. Particle filter based camera tracker fusing marker and feature point based cues. In *Proceedings of the IS&T/SPIE Electronic Imaging Conference on Visual Communications and Image Processing*, San Jose, California, USA, 2007.
18. Uchiyama H, Saito H. Random dot markers. In *IEEE Virtual Reality Conference (VR'11)*, Singapore, March 2011; 35–38.
19. Nakai T, Kise K, Iwamura M. Use of affine invariants in locally likely arrangement hashing for camera-based document image retrieval. In *Lecture Notes in Computer Science (7th International Workshop)*. Springer, Nelson, New Zealand, 2006; 541–552.
20. Wagner D, Reitmayr G, Mulloni A, Drummond T, Schmalstieg D. Real-time detection and tracking for augmented reality on mobile phones. *IEEE Transactions on Visualization and Computer Graphics (TVCG'10)* 2010; **16**: 355–368.
21. Rosten E, Drummond T. Machine learning for high-speed corner detection. In *European Conference on Computer Vision (ECCV'06)*, Vol. 1, Graz, Austria, 2006; 430–443.
22. Gall J, Rosenhahn B, Seidel HP. Robust pose estimation with 3D textured models. In *Pacific-Rim Symposium on Image and Video Technology (PSIVT'06)*, 2006; 84–95.
23. Lowe DG. Distinctive Image features from scale-invariant keypoints. *International Journal on Computer Vision* 2004; **60**: 91–110.
24. Mikolajczyk K, Schmid C. A performance evaluation of local descriptors. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI'05)* 2005; **27**(10): 1615–1630.
25. Lucas BD, Kanade T. An iterative image registration technique with an application to stereo vision. In *International Joint Conference on Artificial Intelligence (IJCAI'81)*, Vancouver, BC, Canada, 1981; 674–679.
26. Fischler MA, Bolles RC. Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM* 1981; **24**(6): 381–395.
27. Chum O, Matas J. Matching with PROSAC - Progressive Sample Consensus. *IEEE Computer Society Conference on Computer Vision and Pattern Recognition* 2005; **1**: 220–226.
28. Lee GA, Billinghurst M, Kim GJ. Occlusion based interaction methods for tangible augmented reality environments. In *Proceedings of ACM SIGGRAPH*

International Conference on Virtual Reality Continuum and Its Applications in Industry (VRCAI'04), Nanyang Technological University, Singapore, 2004; 419–426.

29. McDonald C, Roth G. Replacing a mouse with hand gesture in a plane-based augmented reality system, In *Proceedings of Vision Interface (VI'03)*, Halifax, Canada, 2003.

AUTHORS' BIOGRAPHIES



Hugo Álvarez received his MS degree in Computer Science from the University of the Basque Country in 2007 and his PhD degree in Computer Science from the University of Navarra in 2011. Presently, he is a research staff member at CEIT and a member of the Computer Graphics Group at CEIT. His main research areas include computer vision and augmented reality.



Ibai Leizea received his MS degree in Computer Science from the University of the Basque Country in 2010. He is currently working at the Computer Graphics area of CEIT. His research interest field is focused on the area of computer vision and augmented reality.



Diego Borro is a research staff member at CEIT and an assistant professor at TECNUN (University of Navarra). He received his MS degree in Computer Science from the University of the Basque Country (2000) and his PhD degree in Computer Science from the University of Navarra (2003). He was a visiting scholar (2004) at the Integrated Media Systems Center (IMSC) at the University of Southern California (USC). He is a member of IEEE, ACM, and Eurographics. His main research areas include collision detection, haptic rendering, computer vision, and augmented reality. He is a member of the Haptics Group and the Computer Graphics Group at CEIT.